

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250320>

Outline : Custom models and training with TensorFlow (code [samples](#))

- Definitions / Motivation
 - A Quick Tour of TensorFlow
 - Using TensorFlow like NumPy
 - Customizing Models and Training Algorithms
 - TensorFlow Functions and Graphs
 - Takeaways
-

Source: Aurelien Geron, "Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems", 2019 ([Available online](#)) / **Chapter 12**

Definition / Motivation

Going deeper into TensorFlow and its lower-level Python API.

Why and what for? Extra control for custom loss functions, custom metrics, layers, models, initializers, regularizers, weight constraints, etc.

A Quick Tour of TensorFlow (1)

The most popular deep learning library, open sourced since November 2015.

- The core is very similar to NumPy, but with GPU support.
- Supports distributed computing (across multiple devices and servers).
- Includes a kind of just-in-time (JIT) compiler that allows it to optimize computations for speed and memory usage.
- Computation graphs can be exported to a portable format, so you can train a TensorFlow model in one environment (e.g., using Python on Linux), and run it in another (e.g., using Java on an Android device).
- It implements autodiff, and provides optimizers, such as RMSProp, Nadam and FTRL, so one can easily minimize all sorts of loss functions.

A Quick Tour of TensorFlow (2)

- Offers many more features, built on top of these core features: the most important is [tf.keras](#), but it also data loading & preprocessing ops ([tf.data](#), [tf.io](#), etc.), image processing ops ([tf.image](#)), signal processing ops ([tf.signal](#)), and more.

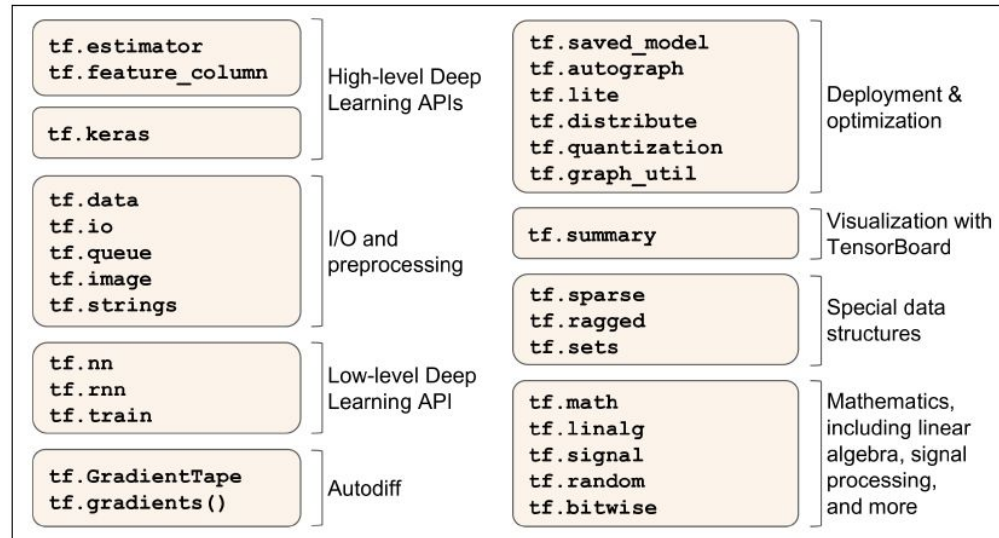


Figure 12-1. TensorFlow's Python API

A Quick Tour of TensorFlow (3)

- At the lowest level, each TensorFlow operation is implemented using highly efficient C++ code.
- Many operations (or *ops* for short) have multiple implementations, called *kernels*: each kernel is dedicated to a specific device type, such as CPUs, GPUs, or even TPUs (Tensor Processing Units).
- GPUs can dramatically speed up computations by splitting computations into many smaller chunks and running them in parallel across many GPU threads.
- TPUs are even faster. You can purchase your own GPU devices, but TPUs are available on [Google Cloud Machine Learning Engine](#).

TensorFlow's architecture

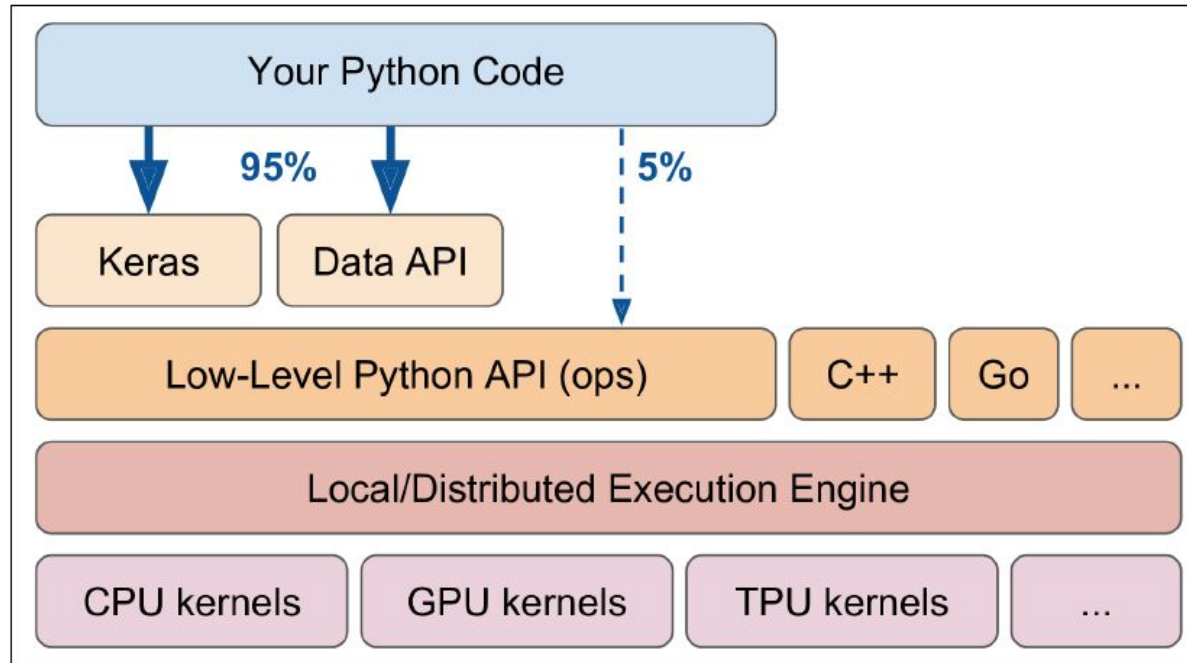


Figure 12-2. TensorFlow's architecture

TensorFlow portability

- Runs on Windows, Linux, and MacOS...
- Also on mobile devices (using TensorFlow Lite), including both iOS and Android.
- In addition to the Python API, there are also C++, Java, Go and Swift APIs.
- And even a Javascript implementation called [TensorFlow.js](#) that makes it possible to run your models directly in your browser.

Using TensorFlow like NumPy

Tensors and Operations

Tensors and NumPy

Type Conversions

Variables

Other Data Structures

Using TensorFlow like NumPy

- TensorFlow's API revolves around **tensors**.
- A tensor is usually a multidimensional array (exactly like a NumPy ndarray), but it can also hold a scalar (a simple value, such as 42).
- The tensors are important when we create custom cost functions, custom metrics, custom layers and more, so let's see how to create and manipulate them.

Tensors and Operations (1)

Matrix, scalar... tf.constant

```
>>> tf.constant([[1., 2., 3.], [4., 5., 6.]]) # matrix  
<tf.Tensor: id=0, shape=(2, 3), dtype=float32, numpy=  
array([[1., 2., 3.],  
       [4., 5., 6.]], dtype=float32)>
```

```
>>> tf.constant(42) # scalar  
<tf.Tensor: id=1, shape=(), dtype=int32, numpy=42>
```

Tensors and Operations (2)

Accessing some of the properties:

```
>>> t = tf.constant([[1., 2., 3.], [4., 5., 6.]])  
>>> t.shape  
TensorShape([2, 3])  
>>> t.dtype  
tf.float32
```

Tensors and Operations (3)

Indexing

```
>>> t[:, 1:]  
<tf.Tensor: id=5, shape=(2, 2), dtype=float32, numpy=  
array([[2., 3.],  
       [5., 6.]], dtype=float32)>  
>>> t[..., 1, tf.newaxis]  
<tf.Tensor: id=15, shape=(2, 1), dtype=float32, numpy=  
array([[2.],  
       [5.]], dtype=float32)>
```

Tensors and Operations (4)

Operations

`tf.add()`

`tf.multiply()`

`tf.square()`

`tf.exp()`

`tf.sqrt()`

...

[tf.math](#)

```
>>> t + 10
<tf.Tensor: id=18, shape=(2, 3), dtype=float32, numpy=
array([[11., 12., 13.],
       [14., 15., 16.]], dtype=float32)>

>>> tf.square(t)
<tf.Tensor: id=20, shape=(2, 3), dtype=float32, numpy=
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)>

>>> t @ tf.transpose(t)
<tf.Tensor: id=24, shape=(2, 2), dtype=float32, numpy=
array([[14., 32.],
       [32., 77.]], dtype=float32)>
```

Tensors and NumPy (1)

Possible to create a tensor from a NumPy array, and vice versa, and to apply TensorFlow operations to NumPy arrays and NumPy operations to tensors.

```
>>> a = np.array([2., 4., 5.])
>>> tf.constant(a)
<tf.Tensor: id=111, shape=(3,), dtype=float64, numpy=array([2., 4., 5.])>
>>> t.numpy() # or np.array(t)
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)
>>> tf.square(a)
<tf.Tensor: id=116, shape=(3,), dtype=float64, numpy=array([4., 16., 25.])>
>>> np.square(t)
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)
```

Tensors and NumPy (2)

- To note that NumPy uses **64-bit precision by default**, while TensorFlow uses **32-bit**.
- 32-bit precision is generally more than enough for neural networks, plus it runs faster and uses less RAM.
- So when you create a tensor from a NumPy array, make sure to set **`dtype=tf.float32`**.

Type Conversions

Type conversions can significantly affect performance. Can be not noticed if is done automatically. So, **TensorFlow does not perform any type conversions automatically**: it just raises an exception if you try to execute an operation on tensors with incompatible types.

```
>>> tf.constant(2.) + tf.constant(40)
Traceback[...]InvalidArgumentError[...]expected to be a float[...]
>>> tf.constant(2.) + tf.constant(40., dtype=tf.float64)
Traceback[...]InvalidArgumentError[...]expected to be a double[...]
```

To convert ([tf.cast\(\)](#)):

```
>>> t2 = tf.constant(40., dtype=tf.float64)
>>> tf.constant(2.0) + tf.cast(t2, tf.float32)
<tf.Tensor: id=136, shape=(), dtype=float32, numpy=42.0>
```

Variables (1)

Constant tensors - not possible to modify, so Variables ([tf.Variable](#)).

Potential use examples: the weights in a neural network need to be tweaked by backpropagation, and other parameters may also need to change over time (e.g., a momentum optimizer keeps track of past gradients).

```
>>> v = tf.Variable([[1., 2., 3.], [4., 5., 6.]])
>>> v
<tf.Variable 'Variable:0' shape=(2, 3) dtype=float32, numpy=
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)>
```

Variables (2)

Possible to perform different operations. Can be modified by assign() method.

```
v.assign(2 * v)          # => [[2., 4., 6.], [8., 10., 12.]]
v[0, 1].assign(42)      # => [[2., 42., 6.], [8., 10., 12.]]
```

You can also modify individual cells (or slices), using the cell's (or slice's) `assign()` method (**direct item assignment will not work**), or using the `scatter_update()` or `scatter_nd_update()` methods:

```
v[:, 2].assign([0., 1.]) # => [[2., 42., 0.], [8., 10., 1.]]
v.scatter_nd_update(indices=[[0, 0], [1, 2]], updates=[100., 200.])
# => [[100., 42., 0.], [8., 10., 200.]]
```

Other Data Structures (1)

- *Sparse tensors* ([tf.SparseTensor](#)) efficiently represent tensors containing mostly 0s. The `tf.sparse` package contains operations for sparse tensors.
- *Tensor arrays* ([tf.TensorArray](#)) are lists of tensors. They have a fixed size by default, but can optionally be made dynamic. All tensors they contain must have the same shape and data type.
- *Ragged tensors* ([tf.RaggedTensor](#)) represent static lists of lists of tensors, where every tensor has the same shape and data type. The `tf.ragged` package contains operations for ragged tensors.

Other Data Structures (2)

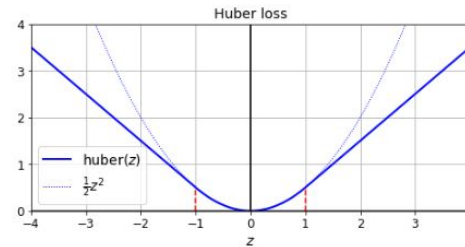
- *String tensors* are regular tensors of type [tf.string](#). These actually represent byte strings, not Unicode strings, so if you create a string tensor using a Unicode string (e.g., a regular Python 3 string like "café"), then it will get encoded to [UTF-8](#) automatically (e.g., b"caf\xc3\xa9"). Alternatively, you can represent Unicode strings using tensors of type `tf.int32`, where each item represents a Unicode codepoint (e.g., [99, 97, 102, 233]). The `tf.strings` package (with an s) contains ops for byte strings and Unicode strings (and to convert one into the other).
- *Sets* are just represented as regular tensors (or sparse tensors) containing one or more sets, and you can manipulate them using operations from the [tf.sets](#) package.
- *Queues*, including First In, First Out (FIFO) queues ([FIFOQueue](#)), queues that can prioritize some items ([PriorityQueue](#)), queues that shuffle their items ([RandomShuffleQueue](#)), and queues that can batch items of different shapes by padding ([PaddingFIFOQueue](#)). These classes are all in the **tf.queue** package.

Customizing Models and Training Algorithms

Custom Loss Functions

Implementing and using own loss functions, e.g. [Huber loss](#).

```
def huber_fn(y_true, y_pred):
    error = y_true - y_pred
    is_small_error = tf.abs(error) < 1
    squared_loss = tf.square(error) / 2
    linear_loss = tf.abs(error) - 0.5
    return tf.where(is_small_error, squared_loss, linear_loss)
```



```
model.compile(loss=huber_fn, optimizer="nadam")
model.fit(X_train, y_train, [...])
```

Saving and Loading Models That Contain Custom Components (1)

When loading a model containing custom objects, you need to map the names to the objects:

```
model = keras.models.load_model("my_model_with_a_custom_loss.h5",
                                custom_objects={"huber_fn": huber_fn})

def create_huber(threshold=1.0):
    def huber_fn(y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < threshold
        squared_loss = tf.square(error) / 2
        linear_loss = threshold * tf.abs(error) - threshold**2 / 2
        return tf.where(is_small_error, squared_loss, linear_loss)
    return huber_fn

model.compile(loss=create_huber(2.0), optimizer="nadam")

model = keras.models.load_model("my_model_with_a_custom_loss_threshold_2.h5",
                                custom_objects={"huber_fn": create_huber(2.0)})
```


Saving and Loading Models That Contain Custom Components (2)

Creating a subclass of the `keras.losses.Loss` class, and implement its `get_config()` method:

```
class HuberLoss(keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)
    def call(self, y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < self.threshold
        squared_loss = tf.square(error) / 2
        linear_loss = self.threshold * tf.abs(error) - self.threshold**2 / 2
        return tf.where(is_small_error, squared_loss, linear_loss)
    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "threshold": self.threshold}
```

```
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="selu", kernel_initializer="lecun_normal",
                        input_shape=input_shape),
    keras.layers.Dense(1),
])
```

```
model.compile(loss=HuberLoss(2.), optimizer="nadam", metrics=["mae"])
```

```
model.fit(X_train_scaled, y_train, epochs=2,
          validation_data=(X_valid_scaled, y_valid))
```

Custom Activation Functions, Initializers, Regularizers, and Constraints (Steps 86-...)

Most of the time, you will just need to write a simple function, with the appropriate inputs and outputs. Examples of a custom activation function, initializer, regularizer, and a custom constraint to ensure weights are all positive:

```
def my_softplus(z): # return value is just tf.nn.softplus(z)
    return tf.math.log(tf.exp(z) + 1.0)

def my_glorot_initializer(shape, dtype=tf.float32):
    stddev = tf.sqrt(2. / (shape[0] + shape[1]))
    return tf.random.normal(shape, stddev=stddev, dtype=dtype)

def my_l1_regularizer(weights):
    return tf.reduce_sum(tf.abs(0.01 * weights))

def my_positive_weights(weights): # return value is just tf.nn.relu(weights)
    return tf.where(weights < 0., tf.zeros_like(weights), weights)
```

Custom Metrics

- Losses and metrics are conceptually different thing:
 - losses are used by Gradient Descent to train a model, so they must be differentiable (at least where they are evaluated) and their gradients should not be 0 everywhere. Plus, it's okay if they are not easily interpretable by humans (e.g. cross-entropy).
 - In contrast, metrics are used to evaluate a model, they must be more easily interpretable, and they can be nondifferentiable or have 0 gradients everywhere (e.g., accuracy).
- In most cases, defining a custom metric function is exactly the same as defining a custom loss function.
 - Huber loss use as a metrics. So, Keras to compute and keep track of its mean since the beginning of the epoch.

```
model.compile(loss="mse", optimizer="nadam", metrics=[create_huber(2.0)])
```

Custom Layers (Steps 134-141)

- Occasionally to build an architecture that contains an “exotic” layer for which TensorFlow does not provide a default implementation. → custom layer.
- Or to build a very repetitive architecture, containing identical blocks of layers repeated many times, and it would be convenient to treat each block of layers as a single layer.

- A, B, C, A, B, C, A, B, C; D=A, B, C, so, D, D, D

```
class MyDense(keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.kernel = self.add_weight(
            name="kernel", shape=[batch_input_shape[-1], self.units],
            initializer="glorot_normal")
        self.bias = self.add_weight(
            name="bias", shape=[self.units], initializer="zeros")
        super().build(batch_input_shape) # must be at the end

    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)

    def compute_output_shape(self, batch_input_shape):
        return tf.TensorShape(batch_input_shape.as_list()[:-1] + [self.units])

    def get_config(self):
        base_config = super().get_config()
        return {**base_config, "units": self.units,
            "activation": keras.activations.serialize(self.activation)}
```

Custom Models

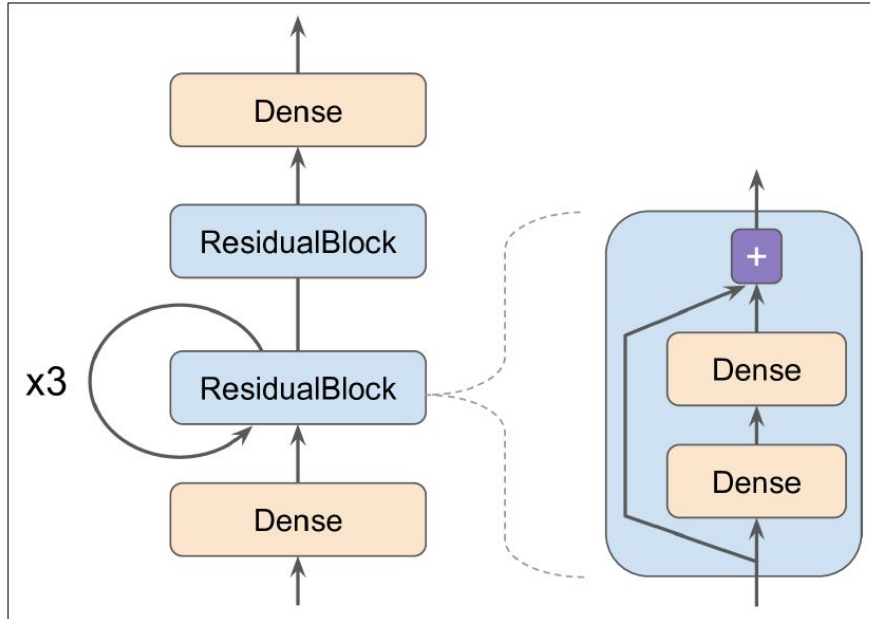


Figure 12-3. Custom Model Example

```
class ResidualBlock(keras.layers.Layer):  
    def __init__(self, n_layers, n_neurons, **kwargs):  
        super().__init__(**kwargs)  
        self.hidden = [keras.layers.Dense(n_neurons, activation="elu",  
                                           kernel_initializer="he_normal")  
                       for _ in range(n_layers)]  
  
    def call(self, inputs):  
        Z = inputs  
        for layer in self.hidden:  
            Z = layer(Z)  
        return inputs + Z
```

Losses and Metrics Based on Model Internals

(Steps 165-167)

- Occasionally to define losses based on other parts of your model, such as the weights or activations of its hidden layers.
- May be useful for regularization purposes, or to monitor some internal aspect of your model.
- Custom loss based on model internals → to compute it based on any part of the model you want.

```
def call(self, inputs, training=None):  
    Z = inputs  
    for layer in self.hidden:  
        Z = layer(Z)  
    reconstruction = self.reconstruct(Z)  
    recon_loss = tf.reduce_mean(tf.square(reconstruction - inputs))  
    self.add_loss(0.05 * recon_loss)  
    if training:  
        result = self.reconstruction_mean(recon_loss)  
        self.add_metric(result)  
    return self.out(Z)
```

Custom Training Loops

- To write your own custom training loops simply to feel more confident that it does precisely what you intend it to do.
- Example: two nested loops: one for the epochs, the other for the batches within an epoch.

```
for epoch in range(1, n_epochs + 1):
    print("Epoch {}/{}".format(epoch, n_epochs))
    for step in range(1, n_steps + 1):
        X_batch, y_batch = random_batch(X_train_scaled, y_train)
        with tf.GradientTape() as tape:
            y_pred = model(X_batch, training=True)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)
        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))
        mean_loss(loss)
        for metric in metrics:
            metric(y_batch, y_pred)
        print_status_bar(step * batch_size, len(y_train), mean_loss, metrics)
    print_status_bar(len(y_train), len(y_train), mean_loss, metrics)
    for metric in [mean_loss] + metrics:
        metric.reset_states()
```

TensorFlow Functions and Graphs

TensorFlow Functions and Graphs (Steps 205-228)

Can speed up your custom code considerably, and it will also make it portable to any platform supported by TensorFlow.

```
def cube(x):
    return x ** 3

>>> cube(2)
8
>>> cube(tf.constant(2.0))
<tf.Tensor: id=18634148, shape=(), dtype=float32, numpy=8.0>
```

```
@tf.function
def tf_cube(x):
    return x ** 3

>>> tf_cube.python_function(2)
8
```

```
>>> tf_cube = tf.function(cube)
>>> tf_cube
<tensorflow.python.eager.def_function.Function at 0x1546fc080>
>>> tf_cube(2)
<tf.Tensor: id=18634201, shape=(), dtype=int32, numpy=8>
>>> tf_cube(tf.constant(2.0))
<tf.Tensor: id=18634211, shape=(), dtype=float32, numpy=8.0>
```

```
result = tf_cube(tf.constant([[7., 8.], [9., 10.], [11., 12.])))
result

<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[ 343.,  512.],
       [ 729., 1000.],
       [1331., 1728.]])>
```

TensorFlow Functions and Graphs (Steps 229-236)

```
@tf.function
def add_10(x):
    for i in range(10):
        x += 1
    return x

add_10(tf.constant(5))

<tf.Tensor: shape=(), dtype=int32, numpy=15>

add_10.get_concrete_function(tf.constant(5)).graph.get_operations()

[<tf.Operation 'x' type=Placeholder>,
 <tf.Operation 'add/y' type=Const>,
 <tf.Operation 'add' type=AddV2>,
 <tf.Operation 'add_1/y' type=Const>,
 <tf.Operation 'add_1' type=AddV2>,
 <tf.Operation 'add_2/y' type=Const>,
 <tf.Operation 'add_2' type=AddV2>,
 <tf.Operation 'add_3/y' type=Const>,
 <tf.Operation 'add_3' type=AddV2>,
 <tf.Operation 'add_4/y' type=Const>,
 <tf.Operation 'add_4' type=AddV2>,
 <tf.Operation 'add_5/y' type=Const>,
 <tf.Operation 'add_5' type=AddV2>,
 <tf.Operation 'add_6/y' type=Const>,
 <tf.Operation 'add_6' type=AddV2>,
 <tf.Operation 'add_7/y' type=Const>,
 <tf.Operation 'add_7' type=AddV2>,
 <tf.Operation 'add_8/y' type=Const>,
 <tf.Operation 'add_8' type=AddV2>,
 <tf.Operation 'add_9/y' type=Const>,
 <tf.Operation 'add_9' type=AddV2>,
 <tf.Operation 'Identity' type=Identity>]
```

TensorFlow Functions and Graphs

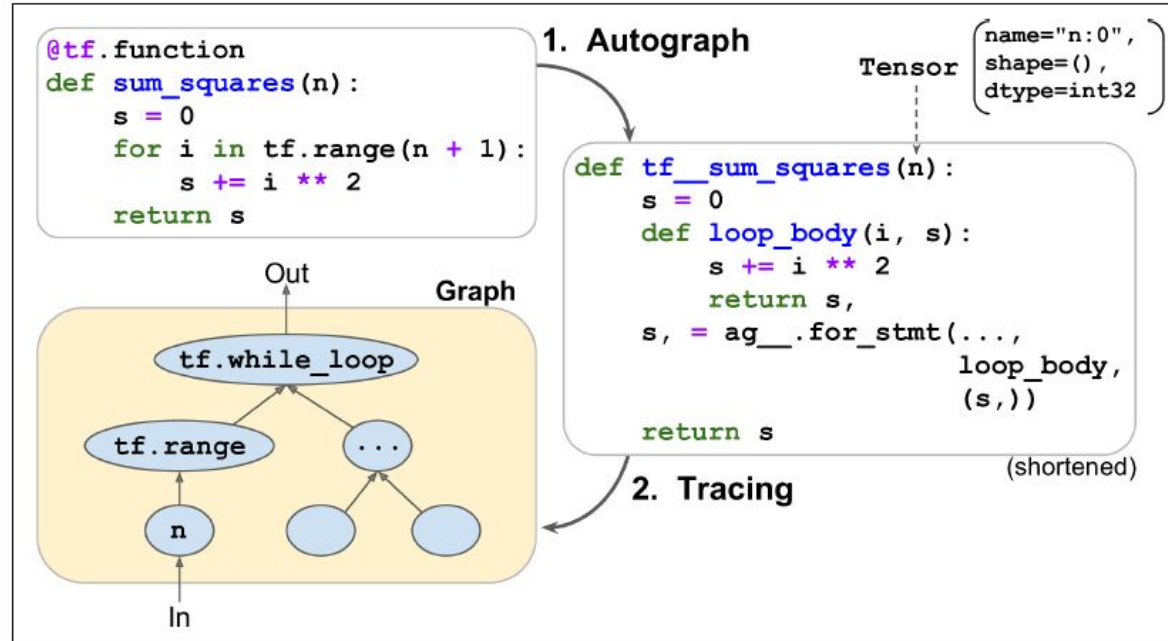


Figure 12-4. How TensorFlow generates graphs using autograph and tracing

Takeaways

- Getting extra control on different elements in the process of building ANN through the customisation of TensorFlow.